

# Project 3: Do better predictions imply better estimation

Helen Bian, Xiaoyi Xu, Kadira Jones

April 16, 2025

## 1 Objective

In order to compare a variety of models with respect to the prediction performance and its corresponding estimator performance, the simulations were conducted based on the simulated data. For each iteration, simple random sampling without replacement (SRWOR) was performed, then the predicted outcome and the estimates based on the proposed estimator for three scenarios were calculated using different models. Consequently, the performance of model prediction and estimator was evaluated through the mean square error respectively.

## 2 Data Generation

To investigate the model performance, the data were generated based on the data generation mechanisms listed below:

$$Y_i = 50 + 10X_{1i} + 2X_{2i} + 5X_{3i} + 7X_{4i} + X_{5i} + 6X_{6i} - 10X_{7i} + 5X_{8i} + 3X_{4i}X_{1i} + X_{1i}X_{2i} + \epsilon_i \quad (1)$$

where  $\epsilon \sim \mathcal{N}(0, 10)$ . Among these covariates,  $X_1, X_5, X_7, X_8$  were generated from different normal distribution,  $X_2$  was from uniform distribution,  $X_3$  was from Poisson distribution, and  $X_4, X_6$  were from different binomial distribution (Please see the attached Rmarkdown for details). Through equation (1), the outcome was generated through the linear combination of each single covariate as well as some of their interactions. The total number of observations was assumed to be  $N = 1,000,000$ . Meanwhile, the non-missing indicator  $r$  with  $r = 0$  denotes the missing, which was generated based on some covariates, which satisfies the missing at random assumption and achieves the average missing rate of 33% by:

$$\text{logit}(r_i) = 1 - X_{2i} + 2X_{4i} + 0.5X_{6i} \quad (2)$$

The simple random sampling without replacement (SRWOR) design was used to select samples with a size of  $n = 10,000$  at each iteration.

## 3 Model

Based on the simulated population data and the specified sampling design described above, the simulation loop was built to obtain the Monte Carlo (MC) estimates evaluating each model's performance in terms of both prediction and its corresponding estimator. The models considered in each scenario, along with the evaluation methods are illustrated below.

### 3.1 Model-assisted estimation

In this scenario, all the covariates were available for the entire population, while the outcome was observed only for the sampled individuals. Therefore, the model was utilized to predict the outcome for the individuals not included in the samples. To evaluate model prediction performance, the MC expectation of the mean squared error (MSE) denoted as  $E_{mc}[mse(y_{ma})]$  was calculated as:

$$E_{mc}\left[\frac{1}{N} \sum_{i \in \mathcal{U}} (y_i - \hat{y}_i)^2\right] \approx \frac{1}{n} \sum_{j=1}^n \left[\frac{1}{N} \sum_{i \in \mathcal{U}} (y_{ij} - \hat{y}_{ij})^2\right] \quad (3)$$

where  $n = 1000$  denotes the number of iterations, and  $N = 1000000$  denotes the size of whole population.

To evaluate the estimator's performance, the MC expectation of the MSE for the estimator denoted as  $E_{mc}[mse(\mu_{ma})]$  was calculated as:

$$E_{mc}[(\mu - \hat{\mu}_j)^2] \approx \frac{1}{n} \sum_{j=1}^n (\mu - \hat{\mu}_j)^2 \quad (4)$$

The model used under this scenario includes linear regression, K-means clustering, neural network, k-nearest neighbor and cubic spline. The linear model was built with the linear combination of each covariate. The K-means algorithm was used to cluster the population into 50 clusters based on all the covariates, and the outcome was then assigned as the average outcome within each cluster. The k-nearest neighbor algorithm is a nonparametric approach used to estimate outcomes by averaging over the five closest observed units in covariate space. The neural network model was developed to predict the the outcome based on a nonlinear combination of covariates, it allows the model to capture complex interactions and nonlinearities between variables. The spline model has an additional part of selecting optimal degrees of freedom for each covariate using AIC. Then, these optimal degrees of freedom for each covariate are used to construct a combined final natural cubic spline model for prediction and estimation.

### 3.2 Imputation

Under this scenario, only sampled data were available. Within the sample, all the covariates were available, while the outcome was only observed for those with  $r = 1$ . Therefore, the model was used to impute the unobserved outcomes. Similarly, the model prediction performance was evaluated through the MC expectation of the mean squared error (MSE) denoted as  $E_{mc}[mse(y_{imp})]$  was calculated as:

$$E_{mc}\left[\frac{1}{|S_m|} \sum_{i \in S_m} (y_i - \hat{y}_i)^2\right] \approx \frac{1}{n} \sum_{j=1}^n \left[\frac{1}{|S_m|} \sum_{i \in S_m} (y_{ij} - \hat{y}_{ij})^2\right] \quad (5)$$

where  $|S_m|$  is the size of the samples with the unobserved outcome. To evaluate the estimator's performance, the MC expectation of the MSE for the estimator denoted as  $E_{mc}[mse(\mu_{imp})]$  was calculated through the same equation as (4).

The model used to impute the missing outcome under this scenario includes linear regression, K-means clustering, neural network, k-nearest neighbor and natural cubic spline as described in section (3.1)

### 3.3 Reweighting

The sample data collected under this scenario is the same as those in the scenario illustrated in 3.2. However, instead of imputing missing outcomes, the samples with observed outcomes were reweighted to represent those with unobserved outcomes, which was assumed to be equivalent to the whole population under certain assumptions. Therefore, the model was used to estimate the probability of being observed. The model prediction performance was evaluated through the MC expectation of the mean squared error (MSE) denoted as  $E_{mc}[mse(y_{rew})]$  was calculated as:

$$E_{mc}\left[\frac{1}{|S_r|} \sum_{i \in S_r} (p_i - \hat{p}_i)^2\right] \approx \frac{1}{n} \sum_{j=1}^n \left[\frac{1}{|S_r|} \sum_{i \in S_r} (p_{ij} - \hat{p}_{ij})^2\right] \quad (6)$$

where  $|S_r|$  is the size of the samples with the observed outcome. To evaluate the estimator's performance, the MC expectation of the MSE for the estimator denoted as  $E_{mc}[mse(\mu_{rew})]$  was calculated through the same equation as (4).

The model used to estimate the response probability includes logistic regression, random forest, neural network and K-nearest neighbor. For the logistic regression, the logit of the response indicator probability  $\text{logit}[P(r = 1)]$  was assumed to be a linear combination of all the covariates. For random forest, 500 trees were used, and the ensemble was used to randomly select covariates for each tree on a different subset of the samples. Each tree predicted a  $r$ , and a majority vote across all trees makes the final prediction of  $r$ .

## 4 Results

Table 1: The MC estimates of the MSE for different models under the model-assisted scenario.

| <b>Model</b>         | $MSE(y_{ma})$ | $MSE(\hat{\mu}_{ma})$ |
|----------------------|---------------|-----------------------|
| Linear regression    | 356.3         | 0.03                  |
| K-means cluster      | 17704         | 1.92                  |
| Neural network       | 34229         | 3.2                   |
| K-nearest neighbor   | 2780          | 0.3                   |
| Natural cubic spline | 982.6         | 0.1                   |

<sup>1</sup> The number of iterations was 1000.

Table 2: The MC estimates of the MSE for different models under the imputation scenario.

| <b>Model</b>         | $MSE(y_{imp})$ | $MSE(\hat{\mu}_{imp})$ |
|----------------------|----------------|------------------------|
| Linear regression    | 363.3          | 19.77                  |
| K-means cluster      | 18796          | 18.06                  |
| Neural network       | 310230         | 111                    |
| K-nearest neighbor   | 322399         | 84                     |
| Natural cubic spline | 988.4          | 27                     |

<sup>1</sup> The number of iterations was 1000.

Table 3: The MC estimates of the MSE for different models under the reweighting scenario.

| <b>Model</b>        | $MSE(p_{rew})$ | $MSE(\hat{\mu}_{rew})$ |
|---------------------|----------------|------------------------|
| Logistic regression | 0.001          | 238.8                  |
| Random forest       | 0.003          | 86.8                   |
| Neural network      | 0.001          | 207                    |
| K-nearest neighbor  | 0.01           | 209                    |

<sup>1</sup> The number of iterations was 1000.

## 5 Discussion

In the model-assisted scenario, the model predictive performance rank from lowest to highest  $MSE(y_{ma})$  as follows: Linear regression, Natural cubic spline, K-nearest neighbor, K-means cluster, and Neural network.  $MSE(\hat{\mu}_{ma})$  of each model are in the same order as above. According to Table 1, it is apparent that the linear regression is the best model in terms of the model prediction and estimation, while the neural network is the poorest.

In the scenario of missing imputation, the model prediction performance rank from lowest to highest  $MSE(y_{imp})$  as follows: Linear regression, Natural cubic spline, K-means cluster, Neural network, and K-nearest neighbor. While the order of the corresponding estimator MSE ( $MSE(\hat{\mu}_{imp})$ ) is different as follows: K-means cluster, Linear regression, Natural cubic spline, K-nearest neighbor, Neural network. From Table 2, we found that the linear regression model has a slightly larger estimation compared to the optimal (K-means cluster), while the linear regression model has a much better prediction. The similar situation also exists between the two worst-performing models, Neural network and K-nearest neighbor. Moreover, the table highlights a clear pattern in the orders of magnitude of  $MSE(y_{imp})$  with the efficiency of estimations. For instance, K-nearest neighbor and Neural network have their  $MSE(y_{imp})$  in the scale of hundreds of thousands, and they yield vastly larger  $MSE(\hat{\mu}_{imp})$  than other models. The Linear regression and cubic spline has their  $MSE(y_{imp})$  in the scale of hundreds, and they output the relatively small  $MSE(\hat{\mu}_{imp})$ . However, there exists an exception, K-means cluster.

In terms of the reweighting estimators, the logistic model, which has the lowest MSE of predicted probability of 0.001, though its performance improvement over the others was minimal. However, it results in the worst estimation for the population mean ( $MSE(\hat{\mu}_{rew}) = 238.8$ ). In contrast, the K-nearest neighbor, which has the worst estimation (0.01) for  $p_{rew}$ , results in a close result (209). The best estimation goes to the random forest model, while its  $MSE(p_{rew})$  is three times the Logistic regression.

## 6 Conclusion

We want to examine this statement: If a model  $\hat{m}_1$  makes better predictions than the other one  $\hat{m}_2$ , it implies that  $\hat{\mu}(\hat{m}_1)$  is going to be a more efficient estimator of  $\hat{\mu}(\hat{m}_2)$ .

For the model-assisted estimator, according to our discussion on the result table, there is a positive relationship between  $MSE(\mu_{ma})$  and  $MSE(y_{ma})$ . This implies that the statement is generally correct in this case.

In terms of the imputed estimator, we found that there is no absolute positive relationship between  $MSE(\mu_{imp})$  and  $MSE(y_{imp})$ , so the statement fails. However, a loose pattern likely remains: when  $MSE(y_{imp})$  jumps by tiers of magnitude, as it does for K-nearest neighbor and the neural network,  $MSE(\mu_{imp})$  results also increase sharply. In other words, a broadly positive association is visible across tiers of model quality, but it will vanish once we zoom in on the cluster of models with comparable predictive accuracy.

Lastly, there is no straight positive relationship found between prediction performance and estimation in reweighting. This statement fails for the reweighted estimators.

# MATH 525 : Project 3 Code

Helen Bian, Kadir Jones and Xiaoyi Xu

2025-04-03

.....Data Generation.....

```
# Setup
set.seed(100)
N<-1000000

# Data Generating Mechanism
# linear combination: generate 5 covariates X1-X8 which are correlated with outcome
X1<-rnorm(N,0,5)
X2<-runif(N,0,10)
X3<-rpois(N,1)
X4<-rbinom(N,1,0.3)
X5<-rnorm(N,20,500)
X6<-rbinom(N,1,0.5)
X7<-rnorm(N,1,10)
X8<-rnorm(N,1,5)

res<-rnorm(N,0,10)
Y=50+10*X1+2*X2+5*X3+7*X4+X5+6*X6-10*X7+5*X8+3*X4*X1+X1*X2+res

pr=plogis(-X2+5*X4+X6+5)      # missing indicator
R<-rbinom(prob=pr,size=1,n=N)  # mean(R)=0.7

dat1<-data.frame(Y,X1,X2,X3,X4,X5,X6,X7,X8,R,pr)
```

Linear Regression, Logistic Regression, K-Means Cluster and Random Forest

```
# Setup
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
## filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
## intersect, setdiff, setequal, union
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.4.3
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##  
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:dplyr':  
##  
## combine
```

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.3
```

```
##  
## Attaching package: 'xgboost'
```

```
## The following object is masked from 'package:dplyr':  
##  
## slice
```

```
library(lightgbm)
```

```
## Warning: package 'lightgbm' was built under R version 4.4.3
```

```
library(nnet)  
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: ggplot2
```

```
##  
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:randomForest':  
##  
## margin
```

```
## Loading required package: lattice
```

```

library(class)

data=dat1
n=1000

# Model
ml<-function(n,data,time){
  mu_ma_l=rep(0,time);mu_ma_km=rep(0,time);
  ymse_ma_l=rep(0,time);ymse_ma_km=rep(0,time);

  mu_imp_l=rep(0,time);mu_imp_km=rep(0,time);
  ymse_imp_l=rep(0,time);ymse_imp_km=rep(0,time);

  mu_ipw_lg=rep(0,time);mu_ipw_rf=rep(0,time);mu_ipw_xg=rep(0,time)
  ymse_ipw_lg=rep(0,time);ymse_ipw_rf=rep(0,time);ymse_ipw_xg=rep(0,time)

  for(i in 1:time){
    row<-sample(nrow(data),n)
    samp<-data[row,]
    not.samp<-data[-row,]
    inc<-n/nrow(data)

    #----- MODEL-ASSISTED -----

    # Linear Regression
    dat.lm=data
    linear.ma<-lm(Y~.-R-pr,data=samp)
    dat.lm$pred.ma<-predict.lm(linear.ma,dat.lm)

    mu_ma_l[i]<-(1/N)*(sum(dat.lm$pred.ma)+sum((dat.lm[row,]$Y-dat.lm[row,]$pred.ma)/inc))
    ymse_ma_l[i]=(sum((data$Y-dat.lm$pred.ma)^2))/nrow(data)

    # k-Means Cluster
    x<-as.matrix(subset(data,selec=-c(Y,R,pr)))
    kmeans_res <- kmeans(x, centers = 50, nstart = 25)
    temp=data.frame(data,cluster=kmeans_res$cluster)
    means_by_cluster <- temp[row, ] %>%
      group_by(cluster) %>%
      summarize(cluster_mean = mean(Y))
    temp <- left_join(temp, means_by_cluster, by = "cluster")

    mu_ma_km[i]<-(1/N)*(sum(temp$cluster_mean)+
      sum((temp[row,]$Y-temp[row,]$cluster_mean)/inc))
    ymse_ma_km[i]=(sum((temp$Y-temp$cluster_mean)^2))/nrow(temp)

    #----- MISSING IMPUTATION -----

    # Linear Regression
    sr=subset(data[row,],R==1)
    sm=subset(data[row,],R==0)
    linear.imp<-lm(Y~.-pr-R,data=sr)
    sm$pred.imp<-predict.lm(linear.imp,sm)
  }
}

```

```

mu_imp_l[i]<-(1/N)*(sum(sr$Y/inc)+sum(sm$pred.imp/inc))
ymse_imp_l[i]=(sum((sm$Y-sm$pred.imp)^2))/nrow(sm)

# K-Means Cluster
x<-as.matrix(subset(samp,selec=-c(Y,R,pr)))
kmeans_res <- kmeans(x, centers = 50, nstart = 25)
temp=data.frame(samp,cluster=kmeans_res$cluster)
means_by_cluster <- temp[R==1, ] %>%
  group_by(cluster) %>%
  summarize(cluster_mean = mean(Y))
temp <- left_join(temp, means_by_cluster, by = "cluster")

mu_imp_km[i]<-(1/N)*(sum(sr$Y/inc)+sum(temp[temp$R==0,"cluster_mean"]/inc))
ymse_imp_km[i]=(sum((sm$Y-temp[temp$R==0,"cluster_mean"]^2))/nrow(sm)

#----- REWEIGHTING -----

# Logistic
p.model<-glm(R~X2+X4+X6,data=samp,family="binomial"(link = "logit"))
sr$p<-predict(p.model,sr,type="response")

mu_ipw_lg[i]<-(sum(sr$Y/(inc*sr$p)))/N
ymse_ipw_lg[i]=(sum((sr$pr-sr$p)^2))/nrow(sr)

# Random Forest
rf_model <- randomForest(as.factor(R) ~ .-Y-pr, data = samp, ntree = 500)
samp$ps_rf <- predict(rf_model, type = "prob")[, "1"]
sr=subset(samp,R==1)
mu_ipw_rf[i]<-(1/N)*(sum(sr$Y/(inc*sr$ps_rf)))
ymse_ipw_rf[i]=(sum((sr$pr-sr$ps_rf)^2))/nrow(sr)

# XGBoost
X <- model.matrix(R ~ . -Y-pr-ps_rf, data = samp)
y <- samp$R
dtrain <- xgb.DMatrix(data = X, label = y)
params <- list(objective = "binary:logistic",eval_metric = "logloss",
  max_depth = 3,eta = 0.1)
model <- xgb.train(params = params,data = dtrain,nrounds = 100,verbose = 0)
ps_xg <- predict(model, newdata = X)
samp$ps_xg <-ps_xg

sr=subset(samp,R==1)
mu_ipw_xg[i]<-(1/N)*(sum(sr$Y/(inc*sr$ps_xg)))
ymse_ipw_xg[i]=(sum((sr$pr-sr$ps_xg)^2))/nrow(sr)
}

out.dat=data.frame(mu_ma_l,mu_ma_km,ymse_ma_l,ymse_ma_km,
  mu_imp_l,mu_imp_km,ymse_imp_l,ymse_imp_km,
  mu_ipw_lg,mu_ipw_rf,mu_ipw_xg,ymse_ipw_lg,ymse_ipw_rf,ymse_ipw_xg)
return(out.dat)
}

```

.....k-Nearest Neighbour and Neural Network.....

```
# Setup
library(nnet)
library(FNN)

## Warning: package 'FNN' was built under R version 4.4.3

##
## Attaching package: 'FNN'

## The following objects are masked from 'package:class':
##
##      knn, knn.cv

# Model
ml_nn_knn <- function(n, data, time) {
  mu_ma_nn = mu_ma_knn = ymse_ma_nn = ymse_ma_knn = rep(0, time)
  mu_imp_nn = mu_imp_knn = ymse_imp_nn = ymse_imp_knn = rep(0, time)
  mu_ipw_nn = mu_ipw_knn = ymse_ipw_nn = ymse_ipw_knn = rep(0, time)

  for (i in 1:time) {
    row <- sample(nrow(data), n)
    samp <- data[row, ]
    not.samp <- data[-row, ]
    inc <- n / nrow(data)

    #----- MODEL-ASSISTED -----
    X_vars <- subset(data, select = -c(Y, R, pr))
    Y_target <- data$Y

    # Neural Network
    nn_model <- nnet(Y ~ ., data = samp[, c("Y", names(X_vars))], size = 5,
                     maxit = 500, linout = TRUE, trace = FALSE)
    pred_nn <- predict(nn_model, newdata = X_vars)

    mu_ma_nn[i] <- (1/N)*(sum(pred_nn) + sum((samp$Y -
                                             predict(nn_model, samp)) / inc))
    ymse_ma_nn[i] <- mean((Y_target - pred_nn)^2)

    # k-Nearest Neighbour
    knn_pred <- knn.reg(train = samp[, names(X_vars)], test = X_vars, y = samp$Y,
                       k = 5)$pred
    mu_ma_knn[i] <- (1/N)*(sum(knn_pred) +
                        sum((samp$Y -knn.reg(train = samp[, names(X_vars)],
                                             test = samp[, names(X_vars)],
                                             y = samp$Y, k = 5)$pred) / inc))
    ymse_ma_knn[i] <- mean((Y_target - knn_pred)^2)

    #----- MISSING IMPUTATION -----
    sr <- subset(samp, R == 1)
    sm <- subset(samp, R == 0)
```

```

# Neural Network
nn_imp <- nnet(Y ~ X2 + X4 + X6, data = sr, size = 5, maxit = 500,
              linout = TRUE, trace = FALSE)
sm$pred_nn <- predict(nn_imp, newdata = sm)

mu_imp_nn[i] <- (1/N)*(sum(sr$Y/inc) + sum(sm$pred_nn/inc))
ymse_imp_nn[i] <- mean((sm$Y - sm$pred_nn)^2)

# k-Nearest Neighbour
if (nrow(sm) > 0 && nrow(sr) > 0) {
  knn_imp <- knn.reg(train = sr[, c("X2", "X4", "X6")], test =
                    sm[, c("X2", "X4", "X6")], y = sr$Y, k = 5)$pred
  mu_imp_knn[i] <- (1/N)*(sum(sr$Y/inc) + sum(knn_imp/inc))
  ymse_imp_knn[i] <- mean((sm$Y - knn_imp)^2)
} else {
  mu_imp_knn[i] <- NA
  ymse_imp_knn[i] <- NA
}

#----- REWEIGHTING -----

# Neural Network
samp$R <- as.factor(samp$R)
nn_ps <- nnet(R ~ X2 + X4 + X6, data = samp, size = 5, maxit = 500, trace = FALSE)
samp$ps_nn <- predict(nn_ps, newdata = samp, type = "raw")
sr <- subset(samp, R == 1)
sr$ps_nn <- samp[samp$R == 1, "ps_nn"]
mu_ipw_nn[i] <- (1/N)*(sum(sr$Y / (inc * sr$ps_nn)))
ymse_ipw_nn[i] <- mean((sr$pr - sr$ps_nn)^2)

# k-Nearest Neighbour
knn_ps <- knn.reg(train = samp[, c("X2", "X4", "X6")],
                  test = samp[, c("X2", "X4", "X6")],
                  y = as.numeric(as.character(samp$R)), k = 5)$pred
sr$ps_knn <- knn_ps[samp$R == 1]
mu_ipw_knn[i] <- (1/N)*(sum(sr$Y / (inc * sr$ps_knn)))
ymse_ipw_knn[i] <- mean((sr$pr - sr$ps_knn)^2)
}

out.dat <- data.frame(
  mu_ma_nn, mu_ma_knn, ymse_ma_nn, ymse_ma_knn,
  mu_imp_nn, mu_imp_knn, ymse_imp_nn, ymse_imp_knn,
  mu_ipw_nn, mu_ipw_knn, ymse_ipw_nn, ymse_ipw_knn
)
return(out.dat)
}

# Execute Code
# out_nn_knn <- ml_nn_knn(10000, dat1, 1000)
# tmu=mean(dat1$Y)

# Summary Outputs
# mean((out_nn_knn$mu_ma_nn-tmu)^2)

```

```

# mean((out_nn_knn$mu_ma_knn-tmu)^2)
# mean(out_nn_knn$ymse_ma_nn)
# mean(out_nn_knn$ymse_ma_knn)

# mean((out_nn_knn$mu_imp_nn-tmu)^2)
# mean(((out_nn_knn$mu_imp_knn, na.rm = TRUE)-tmu)^2)
# mean(out_nn_knn$ymse_imp_nn)
# mean(out_nn_knn$ymse_imp_knn, na.rm = TRUE)

# mean((out_nn_knn$mu_ipw_nn-tmu)^2)
# mean((out_nn_knn$mu_ipw_knn-tmu)^2)
# mean(out_nn_knn$ymse_ipw_nn)
# mean(out_nn_knn$ymse_ipw_knn)

```

.....Cubic Spline .....

```

# Setup
library(splines)
library(sampling)

```

```

##
## Attaching package: 'sampling'

## The following object is masked from 'package:caret':
##
##      cluster

```

```

set.seed(100)
N<-1000000

# Data Generation
X1<-rnorm(N,0,5)
X2<-runif(N,0,10)
X3<-rpois(N,1)
X4<-rbinom(N,1,0.3)
X5<-rnorm(N,20,500)
X6<-rbinom(N,1,0.5)
X7<-rnorm(N,1,10)
X8<-rnorm(N,1,5)

res<-rnorm(N,0,10)
Y=50+10*X1+2*X2+5*X3+7*X4+X5+6*X6-10*X7+5*X8+3*X4*X1+X1*X2+res

pr=plogis(-X2+5*X4+X6+5)      # missing indicator
R<-rbinom(prob=pr,size=1,n=N)  # mean(R)=0.7

dat1<-data.frame(Y,X1,X2,X3,X4,X5,X6,X7,X8,R,pr)

N<-nrow(dat1)
Y<-dat1$Y

```

```

set.seed(100)

spline_methods<-function(n, data, time){
  mu_ma=rep(0,time);mu_imp=rep(0,time);ymse_ma=rep(0,time);ymse_imp=rep(0,time)

  for(i in 1:time){
    row<-sample(nrow(data),n)
    samp<-data[row,]
    not.samp<-data[-row,]

    inc<-n/N

    # Model-assisted
    spline_ma<-lm(Y~ns(X1,1)+ns(X2,6)+ns(X3,2)+ns(X4,1)+ns(X5,1)+ns(X6,1)+ns(X7,1),
                  data = samp)
    dat.lm <- data
    dat.lm$pred.ma <- predict(spline_ma, newdata=dat.lm)

    mu_ma[i]<-(1/N)*(sum(dat.lm$pred.ma)+sum((dat.lm[row,]$Y-dat.lm[row,]$pred.ma)/inc))
    ymse_ma[i]=(sum((data$Y-dat.lm$pred.ma)^2))/nrow(data)

    # Missing Imputation
    sr=subset(data[row,],R==1)
    sm=subset(data[row,],R==0)

    if(nrow(sr) > 0){

      spline.imp <- lm(Y~ns(X1,1)+ns(X2,6)+ns(X3,2)+ns(X4,1)+ns(X5,1)+ns(X6,1)+ns(X7,1),
                      data = sr)

      sm$pred.imp<-predict.lm(spline.imp,sm)
      mu_imp[i]<-(1/N)*(sum(sr$Y/inc)+sum(sm$pred.imp/inc))
      ymse_imp[i]=(sum((sm$Y-sm$pred.imp)^2))/nrow(sm)
    }

  }

  out.dat<-data.frame(mu_ma, ymse_ma, mu_imp, ymse_imp)
  return(out.dat)
}

# out=spline_methods(10000,dat1,1000)
# true_mean <- mean(Y)
# cat(true_mean)

# Model-assist
# MSE of predictor : mean((out$mu_ma-true_mean)^2)
# MSE of estimator : mean(out$ymse_ma)

# Missing impute
# MSE of predictor : mean((out$mu_imp-true_mean)^2)
# MSE of estimator : mean(out$ymse_imp)

```